The Open Source Audit - Git

A Capstone Project for the OSS NGMC Course

Prepared by: Priya Bhadoria
Registration Number: 24BEC10009
Course: Open Source Software (OSS)
Date: 31 March 2026
Software Audited: Git

Table of Content

I. Part A: Origin and Philosophy (Page 3-4)
   A1. The Problem and Solution
   A2. The License
   A3. The Ethics of Open Source
II. Part B: Linux Footprint (Page 4-5)
III. Part C: The FOSS Ecosystem (Page 5)
IV. Part D: Open Source vs Proprietary (Page 5-6)
V. Shell Script Documentation (Page 7-13)
VI. Closing Note (Page 13)
VII. References (Page 13)

Part A - Origin and Philosophy

A1. Why Git Was Created

By the early 2000s, the Linux project had grown into a large global effort, and managing constant code changes was becoming difficult. The team relied on BitKeeper, which worked well but wasn't open source. At first, this wasn't a problem since it was free to use, but in 2005, licensing conflicts led to its withdrawal, leaving the team without a reliable tool. Instead of switching to another proprietary option, Linus Torvalds decided to build his own tool. He believed that something so important should be open source, which eventually led to the creation of Git. Git was designed to be fast and reliable, with a distributed system where every developer has a complete copy of the code along with its history. This allows developers to work independently without depending on a central server. It also supports branching and merging, making it easier to experiment without affecting the main code. Another key feature is how it tracks changes clearly, so it's easy to see what has been modified over time. This is one of the reasons why Git has become a standard tool in modern software development.

A2. Understanding the License

Git is covered by the GNU General Public License version 2.0, which focuses more on user freedom rather than just access.
The philosophy of free software is based on four software freedoms:
1. The freedom to run the program for any purpose.
2. The freedom to study how the program works, by examining its source code.
3. The freedom to modify the program.
4. The freedom to distribute copies of the program

Git supports all of these freedoms, but the GPL also adds an important condition. If someone distributes a modified version of the software, they are required to share those changes with others. In simple terms, a company can use Git, improve it, and even sell it, but they cannot keep their improvements private. This makes it different from licenses like MIT, where developers are allowed to use the code in closed-source projects without sharing any modifications. The choice of the GPL reflects a clear philosophy, it is not just about making software free to use, but also about keeping it open for everyone to improve. Because of this, Git is often seen as an example of the idea "free as in freedom" rather than just "free as in price." It doesn't just remove cost; it ensures that users have the ability to study, modify, and share the software openly.

A3. Ethical Side of Open Source

Open source isn't just about writing code, it's also about how people think about sharing and ownership. A common question is whether all software should be open source. In many ways, open source is beneficial because anyone can take existing code and improve it, which often leads to better software overall.
At the same time, it's understandable why companies don't always open their code. They invest a lot of time and money, so wanting control over their work makes sense. Another thing to consider is companies building products on top of open source software. This happens a lot with tools like Linux and Git. Personally, I think it's fair as long as they contribute something back. If they only take without giving anything in return, it doesn't feel right.
A good way to describe this idea is "standing on the shoulders of giants." In software development, almost everything is built on earlier work. Open source makes that process easier because people can learn from existing code instead of starting from scratch. In the end, this approach helps creativity and innovation grow.

Part B - Linux Footprint

Git is easy to install on most Linux systems using package managers.

For example:

On Ubuntu: `sudo apt install git`
On Fedora: `sudo dnf install git`

After installation, you can verify it using:

```
git --version
which git
```

Git is not a background service. It runs only when you use it through commands.

Important Locations

- `/usr/bin/git` - main executable
- `/usr/lib/git-core` - internal components
- `/etc/gitconfig` - system-level settings
- `~/.gitconfig` - user settings

- `.git/` - repository data

Git runs under the current user instead of as a separate system service, which makes it quite flexible to use. At the same time, this means users have to be careful with permissions, especially when working on shared repositories. Updates are handled through the Linux package manager. When new versions are released by the community, maintainers package them and make them available through official repositories, so users can install updates easily.

Part C - FOSS Ecosystem

Git relies on several underlying tools and libraries like libc, curl, OpenSSL, and zlib, which handle things such as networking, security, and data compression. But what's more interesting is what Git has made possible over time.
Platforms like GitHub and GitLab are built on top of Git and provide features like hosting, collaboration, and automation. Git also plays a big role in modern development practices like continuous integration and continuous deployment, where code is tested and deployed regularly.
Even though Git isn't part of the traditional LAMP stack, it has become an essential tool in web development and project management. Another important point is that Git is no longer maintained by just one person. Although it was created by Linus Torvalds, it is now developed and maintained by a global community.

Part D - Open Source vs Proprietary

Aspect	Git	Proprietary Tools (BitKeeper)
Cost	Free	Paid
Transparency	Open	Closed
Modification	Allowed	Restricted
Support	Community-based	Vendor-based
Control	Distributed	Centralized

Final Thoughts

Git has proven to be a reliable and flexible version control system. It works well for both small projects and large development teams. While it can feel a bit confusing at first, especially for beginners, its advantages become clear with regular use.
Compared to other version control systems, Git offers more freedom and openness. Even though it doesn’t always come with official support, its strong community makes up for that by providing help and continuous improvements.
Overall, Git is more than just a tool for version control, it represents a major shift in how software is developed and shared. Based on this, I would definitely recommend using Git in practical scenarios and would also be interested in contributing to its development in the future.

Shell Scripts

Script 1 - System Identity

```
#!/bin/bash

# Script 1: System Identity
# Displays basic system information along with selected software details

# User defined variables
STUDENT_NAME="Priya Bhadoria"
SOFTWARE_CHOICE="Git"

# Fetch system information
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

# Extract Linux distribution name from os-release file
echo "====================================="
echo " Open Source Audit - $STUDENT_NAME"
echo "====================================="
echo "Software : $SOFTWARE_CHOICE"
echo "Distro : $DISTRO"
echo "Kernel Version : $KERNEL"
echo "User : $USER_NAME"
echo "Uptime : $UPTIME"
echo "Date & Time : $DATE"
echo "License : GNU GPL (Linux Kernel)"
```

```
❯ ./script1.sh
=====================================
 Open Source Audit - Priya Bhadoria
=====================================
Software : Git
Distro : Fedora Linux 43 (KDE Plasma Desktop Edition)
Kernel Version : 6.19.9-200.fc43.x86_64
User : priya
Uptime : up 48 minutes
Date & Time : Tuesday 31 March 2026 02:55:11 PM IST
License : GNU GPL (Linux Kernel)
```

Script 2 - Package Inspector

```bash
#!/bin/bash

# Script 2: FOSS Package Inspector
# Checks if a package is installed, shows its details, and prints a short
description

# Take user input for package name
read -p "Enter package name (e.g. git, httpd, mysql, vlc, firefox): " PACKAGE

# Search for matching package in RPM database (case-insensitive)
PKG=$(rpm -qa | grep -i "^$PACKAGE")

# Check if package exists
if [ -n "$PKG" ]; then
   echo "$PACKAGE is installed."

   # Display package details (Name, Version, Summary, License)
   rpm -qi "$PKG" | grep -E "Name|Version|Summary|License"
else
   echo "$PACKAGE is NOT installed."
fi

# Case statement to print a short description of the package
case $PACKAGE in
   git) echo "Git: distributed version control system for tracking code
changes" ;;
   httpd) echo "Apache: web server that powers many websites" ;;
   mysql) echo "MySQL: open source database system" ;;
   vlc) echo "VLC: free media player supporting many formats" ;;
```

```
   firefox) echo "Firefox: open source browser focused on privacy" ;;
   *) echo "No description available" ;;
esac
```

```
❯ ./script2.sh
Enter package name: (e.g. git, httpd, mysql, vlc, firefox):git
git is installed.
Name        : git-core
Version     : 2.53.0
License     : BSD-3-Clause AND GPL-2.0-only AND GPL-2.0-or-later AND LGPL-2.1-or-later AND
Summary     : Core package of git with minimal functionality
Git: distributed version control system for tracking code changes
```

Script 3 - Disk and Permission Auditor

```
#!/bin/bash

# Script 3: Disk and Permission Auditor
# Loops through important system directories and displays their size,
# permissions, owner, and group information

# List of directories to check
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

# Loop through each directory
for DIR in "${DIRS[@]}"; do
    # Check if directory exists
    if [ -d "$DIR" ]; then
        # Get permissions, owner, and group using ls and awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # Get disk usage (human-readable), suppress errors if access is
denied
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        # Display result
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
```

```
    else
        # Handle case where directory does not exist
        echo "$DIR does not exist on this system"
    fi
done

echo ""
echo "Checking Git config files..."
echo "----------------------------"

# Flag to check if any config file is found
FOUND=0

# Check common Git config locations (system + user)
for FILE in "/etc/gitconfig" "$HOME/.gitconfig"; do
    if [ -f "$FILE" ]; then
        # Display permissions, owner, group, and file name
        ls -l "$FILE" | awk '{print $1, $3, $4, $9}'
        FOUND=1
    fi
done

# If no config files found
if [ $FOUND -eq 0 ]; then
    echo "No Git config files found"
fi
```

```
❯ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x. root root | Size: 37M
/var/log => Permissions: drwxr-xr-x. root root | Size: 208M
/home => Permissions: drwxr-xr-x. root root | Size: 2.4G
/usr/bin => Permissions: dr-xr-xr-x. root root | Size: 553M
/tmp => Permissions: drwxrwxrwt. root root | Size: 144K

Checking Git config files...
----------------------------
No Git config files found
```

Script 4 - Log Analyzer

```bash
#!/bin/bash

# Script 4: Log File Analyzer
# Usage: ./script4.sh logfile [keyword]
# Reads a log file, counts keyword occurrences, and shows recent matches

# Assign command-line arguments
LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error'
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
   echo "Error: File $LOGFILE not found."
   exit 1
fi

# Do-while style loop to handle empty file
while true; do
   if [ ! -s "$LOGFILE" ]; then
       echo "File is empty. Enter another file:"
       read LOGFILE

       # Re-check if new file exists
       if [ ! -f "$LOGFILE" ]; then
           echo "Error: File not found."
           continue
       fi
   else
       break
   fi
done

# Read file line by line and count keyword matches
while IFS= read -r LINE; do
   if echo "$LINE" | grep -iq "$KEYWORD"; then
       COUNT=$((COUNT + 1))   # Increment counter
   fi
done < "$LOGFILE"

# Print summary
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
```

```
# Display last 5 matching lines
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
❯ ./script4.sh /var/log/dnf.log error
Keyword 'error' found 0 times in /var/log/dnf.log

Last 5 matching lines:
```

Script 5 - Manifesto Generator

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
# Collects user input and generates a personalized manifesto saved to a
file

echo "Answer three questions to generate your manifesto."
echo ""

# Take user input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Get current date in readable format
DATE=$(date '+%d %B %Y')

# Create output filename using current username
OUTPUT="manifesto_$(whoami).txt"

# Write manifesto content to file (overwrite if exists)
echo "Open Source Manifesto - $DATE" > "$OUTPUT"
echo "" >> "$OUTPUT"

# Append user responses to build the manifesto
echo "I use $TOOL in my daily work." >> "$OUTPUT"
echo "For me, freedom means $FREEDOM." >> "$OUTPUT"
echo "I believe in sharing knowledge, and I would build $BUILD and make it
available to everyone." >> "$OUTPUT"
```

```
# Display confirmation and show file content
echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat "$OUTPUT"
```

```
./script5.sh
```

```
Scripts 〉 manifesto_priya.txt
1  Open Source Manifesto - 31 March 2026
2
3  I use git in my daily work.
4  For me, freedom means freedom.
5  I believe in sharing knowledge, and I would build General Artificial Intelligence like JARVIS and make it available to everyone
6
```

Closing Note

This project demonstrates that open source means not only coding, but also the sharing of responsibilities and knowledge. Git is a prime example of how a solution, initially created to solve a particular problem, can grow and shape an entire industry.

References

Git Documentation. (n.d.). Retrieved from https://git-scm.com/docs
GNU Project. (n.d.). GNU General Public License v2. Retrieved from https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
Open Source Initiative. (n.d.). Retrieved from https://opensource.org/
GitHub. (n.d.). Retrieved from https://github.com/about
Ubuntu Documentation. (n.d.). Retrieved from https://ubuntu.com/tutorials/install-git
Wikipedia contributors. (n.d.). Git. In Wikipedia. Retrieved from https://en.wikipedia.org/wiki/Git